

**RAFAEL FABIAN CARREÑO BECERRA
ALONSO ANDRES HENAO ROA**

Programación II

SISTEMA DE TICKETS Y SOPORTE AL CLIENTE

Documentación de Modelamiento de Software

Quiz 3 — Aplicación Cliente-Servidor
Tecnologías: Java • SQL • Apache NetBeans
Arquetipo: MVC (Modelo-Vista-Controlador)

2026

1. Definición del Problema

1.1 Descripción General

Las empresas que ofrecen soporte al cliente frecuentemente enfrentan dificultades para gestionar, priorizar y hacer seguimiento de las solicitudes entrantes. Sin un sistema

centralizado, los tickets pueden perderse, duplicarse o atenderse fuera de orden, generando insatisfacción en los clientes y sobrecarga en los agentes de soporte.

El sistema propuesto busca resolver este problema mediante una aplicación cliente-servidor desarrollada en Java con persistencia en una base de datos Oracle (SQL*Plus), que permita registrar, asignar, escalar y cerrar tickets de soporte de manera ordenada y trazable.

1.2 Alcance del Sistema

- Registro de clientes y agentes de soporte.
- Creación de tickets con categoría, prioridad y descripción.
- Asignación de tickets a agentes disponibles.
- Seguimiento del estado del ticket: Abierto → En Progreso → Resuelto → Cerrado.
- Generación de reportes a través de VIEWS en SQL.
- Interfaz gráfica de escritorio desarrollada con Java Swing en NetBeans.

1.3 Problemas Identificados

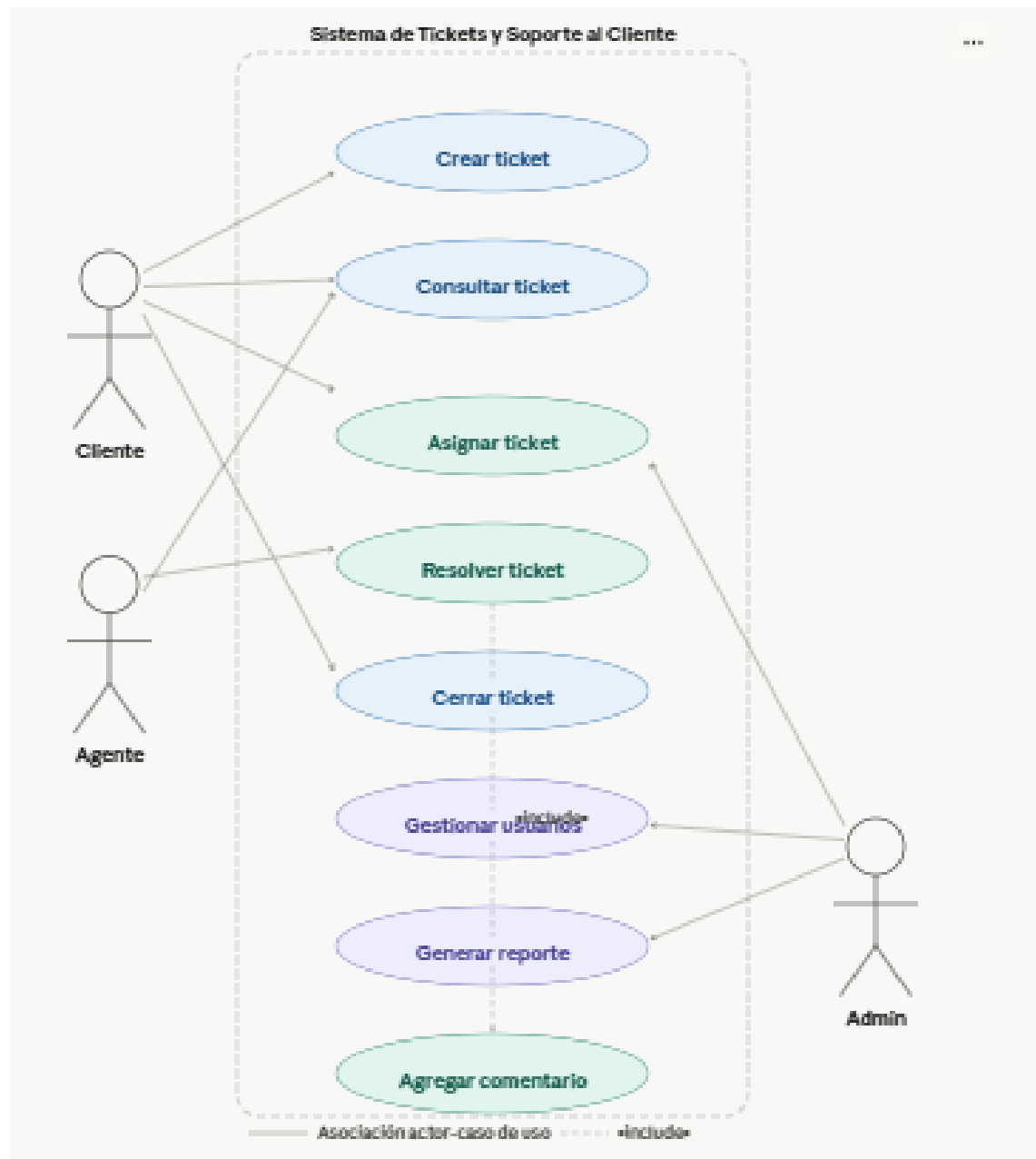
#	Problema	Impacto
1	No existe un canal unificado para reportar problemas técnicos.	Alto
2	Los tickets no tienen seguimiento ni historial de cambios.	Alto
3	Los agentes no saben qué solicitudes tienen mayor prioridad.	Medio
4	No hay reportes de tiempos de resolución ni carga de trabajo.	Medio
5	La comunicación cliente-agente es informal e ineficiente.	Bajo

2. Casos de Uso

2.1 Actores del Sistema

Actor	Descripción	Rol
Cliente	Usuario que reporta un problema o solicita asistencia.	Externo
Agente de Soporte	Técnico que atiende y resuelve los tickets asignados.	Interno
Administrador	Gestiona usuarios, categorías y genera reportes globales.	Interno
Sistema (BD Oracle)	Persiste la información y ejecuta las vistas SQL.	Sistema

2.2 Diagrama de Casos de Uso A continuación se describen los casos de uso principales del sistema:



CU-01: Crear Ticket

Campo	Descripción
Actor principal	Cliente
Precondición	El cliente debe estar registrado e iniciado sesión.
Flujo principal	1. El cliente accede al módulo de tickets. 2. Selecciona «Nuevo Ticket». 3. Completa: asunto, descripción, categoría y prioridad. 4. El sistema genera un ID único y registra el ticket con estado «Abierto». 5. Se notifica al agente disponible.

Postcondición	El ticket queda registrado en la base de datos con estado ABIERTO.
Flujo alternativo	Si los campos están incompletos, el sistema muestra un mensaje de error.

CU-02: Asignar Ticket

Campo	Descripción
Actor principal	Administrador / Sistema
Precondición	Debe existir al menos un ticket con estado ABIERTO y un agente disponible.
Flujo principal	1. El administrador visualiza la lista de tickets pendientes. 2. Selecciona un ticket. 3. Elige un agente de soporte del listado. 4. El sistema actualiza el estado a «En Progreso» y registra la asignación.
Postcondición	El ticket queda asignado al agente y su estado cambia a EN_PROGRESO.
Flujo alternativo	Si no hay agentes disponibles, el ticket permanece en cola ABIERTO.

CU-03: Resolver Ticket

Campo	Descripción
Actor principal	Agente de Soporte
Precondición	El agente tiene al menos un ticket asignado en estado EN_PROGRESO.
Flujo principal	1. El agente abre el ticket asignado. 2. Registra las acciones realizadas en el campo «Comentarios». 3. Cambia el estado a «Resuelto». 4. El sistema registra la fecha y hora de resolución.
Postcondición	El ticket queda en estado RESUELTO con comentarios y fecha registrados.
Flujo alternativo	El agente puede escalar el ticket al administrador si supera sus capacidades.

CU-04: Cerrar Ticket

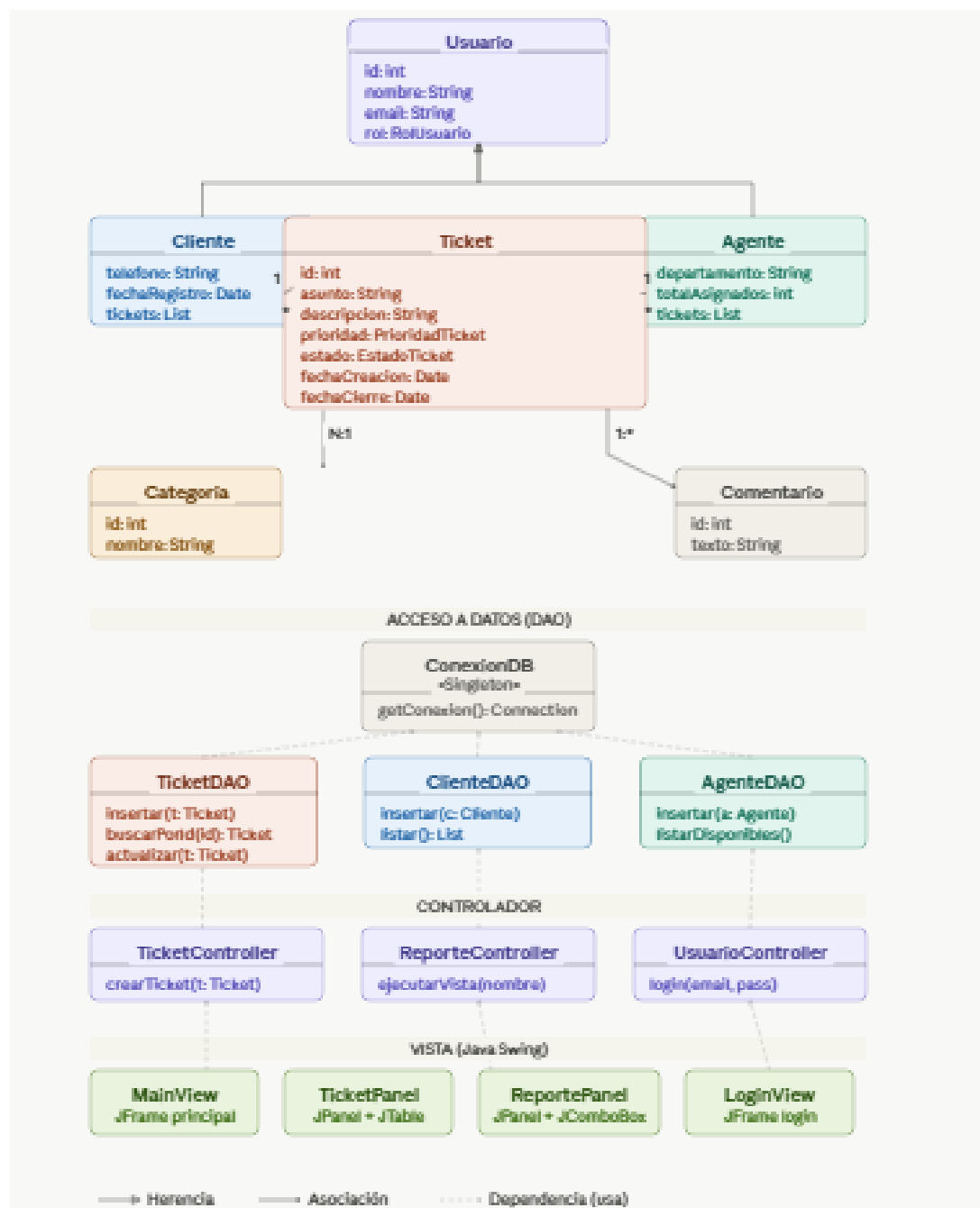
Campo	Descripción
Actor principal	Cliente / Administrador

Precondición	El ticket debe estar en estado RESUELTO.
Flujo principal	1. El cliente o administrador revisa la resolución. 2. Confirma que el problema fue solucionado. 3. El sistema cambia el estado a «Cerrado» y registra la fecha de cierre.
Postcondición	El ticket queda CERRADO. No puede modificarse.
Flujo alternativo	Si el cliente no está conforme, puede reabrir el ticket.

CU-05: Generar Reporte

Campo	Descripción
Actor principal	Administrador
Precondición	Debe haber al menos un ticket registrado en el sistema.
Flujo principal	1. El administrador accede al módulo de reportes. 2. Selecciona el tipo de reporte (por estado, por agente, por categoría, por prioridad). 3. El sistema invoca la VIEW correspondiente en Oracle. 4. Se muestra el reporte en pantalla con opción de exportar.
Postcondición	El administrador visualiza el reporte solicitado.
Flujo alternativo	Si no hay datos, el sistema muestra «Sin registros».

3. Diagrama de Clases



3.1 Descripción de Clases Principales

El sistema sigue el patrón MVC. Las clases del modelo representan las entidades de la base de datos:

Clase	Paquete	Atributos Principales	Responsabilidad
Usuario	modelo	id, nombre, email, contraseña, rol	Entidad base para Cliente y Agente

Cliente	modelo	id, nombre, email, telefono, fechaRegistro	Extiende Usuario. Crea tickets.
Agente	modelo	id, nombre, departamento, ticketsAsignados	Extiende Usuario. Resuelve tickets.
Ticket	modelo	id, asunto, descripcion, prioridad, estado, fechaCreacion, fechaCierre	Entidad principal del sistema.
Categoria	modelo	id, nombre, descripcion	Clasifica los tickets.
Comentario	modelo	id, texto, fecha, idTicket, idUsuario	Historial de acciones sobre el ticket.
TicketDAO	dao	conn: Connection	Operaciones CRUD para Ticket en Oracle.
ClienteDAO	dao	conn: Connection	Operaciones CRUD para Cliente.
AgenteDAO	dao	conn: Connection	Operaciones CRUD para Agente.
ConexionDB	util	url, user, pass	Singleton para la conexión JDBC a Oracle.
TicketController	controlador	ticketDAO, clienteDAO	Lógica de negocio del módulo tickets.
ReporteController	controlador	conn: Connection	Invoca las VIEWS y retorna ResultSet.
MainView	vista	frame, panels	JFrame principal con navegación por paneles.
TicketPanel	vista	table, form	JPanel para listar y crear tickets (Swing).
ReportePanel	vista	table, comboBox	JPanel para visualizar reportes desde VIEWS.

3.2 Relaciones entre Clases

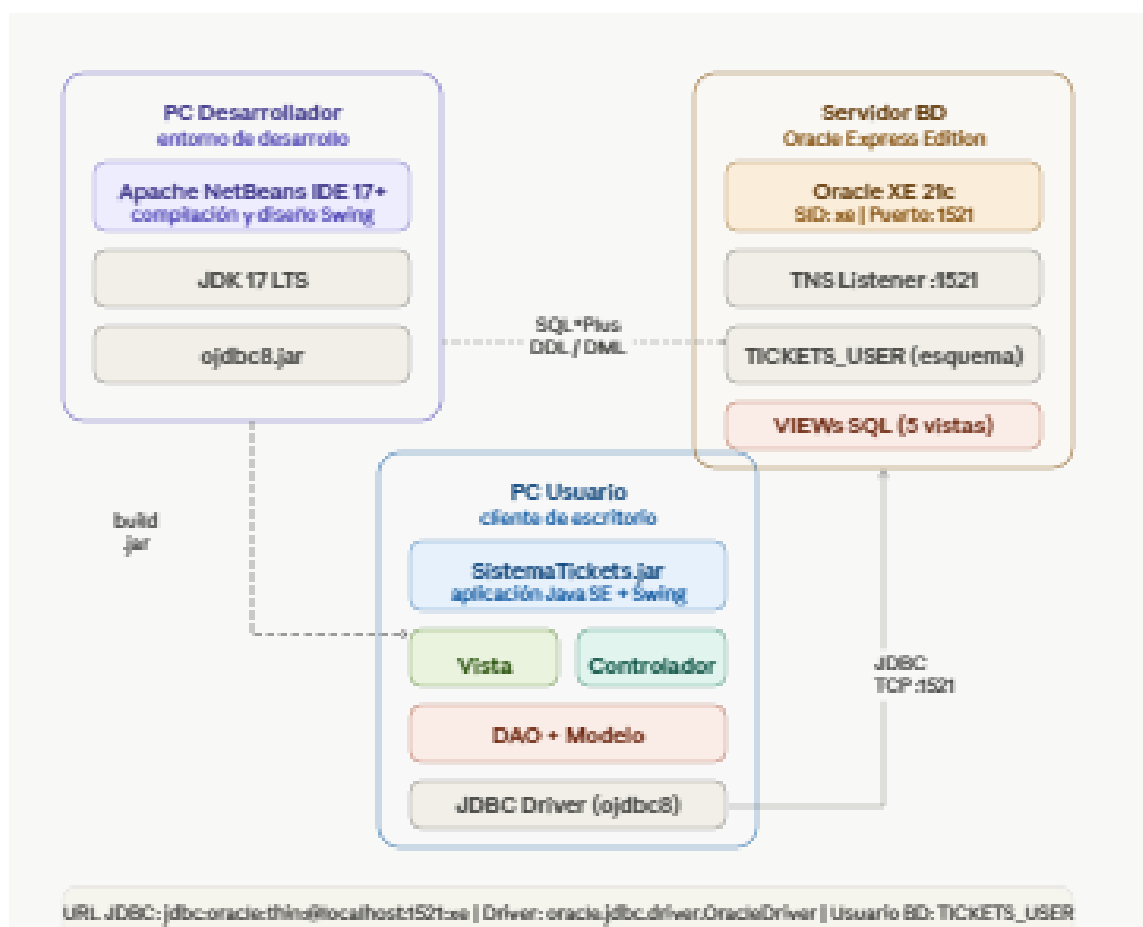
- Cliente — Ticket: Uno a muchos (un cliente puede tener múltiples tickets).
- Agente — Ticket: Uno a muchos (un agente puede atender múltiples tickets).
- Ticket — Categoria: Muchos a uno (varios tickets pertenecen a una categoría).
- Ticket — Comentario: Uno a muchos (un ticket puede tener varios comentarios).
- TicketController usa TicketDAO, ClienteDAO (dependencia).

- ConexionDB es Singleton: todas las clases DAO la invocan para obtener la conexión.
- MainView contiene TicketPanel y ReportePanel (composición).

3.3 Enumeraciones

Enumeración	Valores
EstadoTicket	ABIERTO EN_PROGRESO RESUELTO CERRADO CANCELADO
PrioridadTicket	BAJA MEDIA ALTA CRITICA
RolUsuario	CLIENTE AGENTE ADMINISTRADOR

4. Diagrama de Despliegue



4.1 Descripción de la Arquitectura

La aplicación sigue una arquitectura cliente-servidor de dos capas (2-tier), donde el cliente ejecuta la lógica de presentación y negocio, y el servidor aloja la base de datos Oracle.

Nodo	Componente	Tecnología	Responsabilidad
Máquina del Cliente	Aplicación Java (JAR)	Java SE + Swing	Interfaz gráfica + lógica de negocio
Máquina del Cliente	JDBC Driver	ojdbc8.jar	Comunicación con Oracle
Servidor de Base de Datos	Oracle DB / SQL*Plus	Oracle 11g/19c	Persistencia, VIEWS, procedimientos
Servidor de Base de Datos	Listener Oracle	TNS / Port 1521	Acepta conexiones remotas JDBC
Entorno de Desarrollo	Apache NetBeans IDE	NetBeans 17+	Desarrollo, compilación y pruebas

4.2 Flujo de Comunicación

1. El usuario interactúa con la interfaz Swing (MainView → TicketPanel).
2. La vista llama al controlador correspondiente (TicketController).
3. El controlador invoca el DAO (TicketDAO) que construye la sentencia SQL.
4. ConexionDB provee la conexión JDBC al servidor Oracle (puerto 1521).
5. Oracle procesa la consulta, ejecuta triggers o VIEWS si aplica, y devuelve el ResultSet.
6. El DAO convierte el ResultSet en objetos Java (modelo) y los retorna al controlador.
7. El controlador actualiza el estado en la vista (tabla Swing).

4.3 Protocolo de Conexión

Parámetro	Valor
Driver JDBC	oracle.jdbc.driver.OracleDriver
URL de conexión	jdbc:oracle:thin:@localhost:1521:xe
Puerto	1521 (TNS Listener)
SID / Service	xe (Oracle Express Edition)
Usuario BD	TICKETS_USER (esquema dedicado)

5. Interfaces de Usuario

5.1 Pantallas del Sistema

Las interfaces se desarrollan con Java Swing en NetBeans, usando JFrame, JPanel, JTable, JComboBox y JButton como componentes principales.

Pantalla	Descripción	Componentes Swing
Login	Autenticación de usuario con email y contraseña. Redirige según rol.	TextField, JPasswordField, JButton
Panel Principal (MainView)	Ventana base con menú lateral para navegar entre módulos.	JFrame, JMenuBar, JSplitPane
Lista de Tickets	Tabla con todos los tickets: ID, asunto, prioridad, estado, agente asignado. Permite filtrar.	JTable, DefaultTableModel, JComboBox
Formulario Nuevo Ticket	Formulario para crear un ticket. Campos: asunto, descripción, categoría, prioridad.	TextField, JTextArea, JComboBox, JButton
Detalle de Ticket	Vista completa del ticket con historial de comentarios y botones de cambio de estado.	JTextArea, JScrollPane, JButton
Asignación de Ticket	Permite al administrador asignar un ticket a un agente disponible.	JComboBox (agentes), JButton
Panel de Reportes	Selector de tipo de reporte + tabla de resultados obtenidos desde VIEWS SQL.	JComboBox, JTable, JButton (Exportar)
Gestión de Usuarios	CRUD para clientes y agentes. Solo accesible para el administrador.	JTable, JDialog (formulario), JButton

5.2 Descripción de la Pantalla Principal — Lista de Tickets

La pantalla más importante del sistema es la lista de tickets. Contiene:

- Barra superior: logo del sistema y nombre del usuario autenticado.
- Panel izquierdo: menú de navegación con iconos (Tickets, Mis tickets, Reportes, Usuarios, Configuración).
- Área central: JTable con columnas: ID | Asunto | Cliente | Agente | Prioridad | Estado | Fecha Creación.

- Panel de filtros: JComboBox para filtrar por estado (Todos/Abierto/En Progreso/Resuelto/Cerrado) y por prioridad.
- Barra inferior: botones «Nuevo Ticket», «Asignar», «Ver Detalle», «Cerrar Ticket».

5.3 Criterios de Diseño UI

Criterio	Decisión de Diseño
Paleta de colores	Azul corporativo (#2C5F8A) para encabezados; blanco para fondos; rojo para prioridad CRÍTICA; verde para RESUELTO.
Fuente	Arial 12pt para tablas; Arial 14pt para títulos de panel.
Accesibilidad	Todos los botones tienen tooltips descriptivos. Mensajes de error en JOptionPane.
Validación	Validación de campos vacíos y formatos de email en el lado cliente antes de enviar al DAO.
Feedback	Barra de estado inferior que muestra la última acción realizada (ej: «Ticket #42 asignado a Juan Pérez»).

6. Modelo de Base de Datos

6.1 Tablas Principales

Tabla	Columnas Principales	Descripción
USUARIOS	id_usuario PK, nombre, email UNIQUE, password, rol	Almacena todos los usuarios del sistema.
CLIENTES	id_cliente PK, id_usuario FK, telefono, fecha_registro	Datos adicionales de clientes.
AGENTES	id_agente PK, id_usuario FK, departamento	Datos adicionales de agentes de soporte.
CATEGORIAS	id_categoria PK, nombre, descripcion	Categorías de tickets (Red, Hardware, Software, etc).
TICKETS	id_ticket PK, asunto, descripcion, prioridad, estado, id_cliente FK, id_agente FK, id_categoria FK, fecha_creacion, fecha_cierre	Tabla central del sistema.

COMENTARIOS	id_comentario PK, id_ticket FK, id_usuario FK, texto, fecha_comentario	Historial de acciones sobre cada ticket.
-------------	--	--

6.2 Vistas SQL (VIEW) para Reportes

Se crean las siguientes VIEWS en Oracle para generar reportes sin necesidad de construir queries complejos en Java:

Vista SQL	Propósito	Columnas Principales
VW_TICKETS_ACTIVOS	Todos los tickets que no están cerrados ni cancelados.	id_ticket, asunto, estado, prioridad, nombre_cliente, nombre_agente
VW_CARGA_AGENTES	Número de tickets asignados por agente, para balancear carga.	id_agente, nombre_agente, total_tickets, tickets_en_progreso
VW_TICKETS_POR_CATEGORIA	Distribución de tickets por categoría.	nombre_categoria, total_tickets, tickets_resueltos, tickets_abiertos
VW_TIEMPOS_RESOLUCION	Tiempo promedio de resolución por agente (fecha_cierre - fecha_creacion).	nombre_agente, promedio_horas, total_resueltos
VW_REPORTES_MENSUAL	Resumen mensual: tickets creados, resueltos y cerrados por mes.	anio, mes, total_creados, total_resueltos, total_cerrados

7. Arquetipo Utilizado: MVC

7.1 Patrón Modelo-Vista-Controlador (MVC)

El arquetipo elegido para este proyecto es el patrón de arquitectura MVC (Modelo-Vista-Controlador), que es el estándar para aplicaciones de escritorio desarrolladas en Java con NetBeans.

Capa	Responsabilidad	Tecnología / Paquete
------	-----------------	----------------------

Modelo (Model)	Representa las entidades del dominio y el acceso a datos. Incluye las clases POJO (Ticket, Cliente, Agente) y las clases DAO que ejecutan SQL sobre Oracle.	Java + JDBC paquete: modelo, dao
Vista (View)	Interfaz gráfica del usuario. Implementada con Java Swing usando JFrame, JPanel, JTable. No contiene lógica de negocio.	Java Swing paquete: vista
Controlador (Controller)	Intermediario entre Vista y Modelo. Recibe eventos de la vista, invoca la lógica del DAO y actualiza la vista con los resultados.	Java puro paquete: controlador

7.2 Justificación de la Elección

- Separación de responsabilidades: cada capa puede modificarse independientemente. Si cambia la UI, el modelo no se ve afectado.
- Facilita el trabajo en equipo: un integrante trabaja en la vista (Swing) mientras el otro trabaja en el modelo/DAO (Java + SQL) sin conflictos.
- Compatible con NetBeans: el IDE tiene soporte nativo para diseño visual de JFrames bajo MVC.
- Escalabilidad: si en el futuro se desea migrar la vista a una aplicación web, solo se reemplaza la capa Vista manteniendo el Modelo y Controlador intactos.
- Mantenibilidad: es el patrón más documentado para aplicaciones Java de escritorio, lo que facilita el mantenimiento y la comprensión del código.

7.3 Estructura de Paquetes en NetBeans

Paquete	Contenido
co.sistematickets.modelo	Ticket.java, Cliente.java, Agente.java, Categoria.java, Comentario.java
co.sistematickets.dao	TicketDAO.java, ClienteDAO.java, AgenteDAO.java, CategoriaDAO.java
co.sistematickets.controlador	TicketController.java, UsuarioController.java, ReporteController.java
co.sistematickets.vista	MainView.java, LoginView.java, TicketPanel.java, ReportePanel.java
co.sistematickets.util	ConexionDB.java, Constantes.java, ValidadorUtil.java

8. Evidencia del Despliegue

8.1 Requisitos del Entorno

Componente	Versión Recomendada	Notas
Java Development Kit (JDK)	JDK 17 LTS o superior	Variable JAVA_HOME configurada en el sistema.
Apache NetBeans IDE	NetBeans 17 o 19	Con soporte para Java SE y diseñador Swing integrado.
Oracle Database Express Edition	Oracle XE 21c	Servicio OracleServiceXE y OracleXETNSListener activos.
JDBC Driver Oracle	ojdbc8.jar (Oracle 19c+)	Agregar al classpath del proyecto en NetBeans.
SQL*Plus	Incluido con Oracle XE	Para ejecutar los scripts DDL/DML de creación de tablas y vistas.

8.2 Pasos para el Despliegue

Paso 1 — Configuración de la Base de Datos:

- Iniciar SQL*Plus como SYSDBA: `sqlplus sys/password@xe as sysdba`
- Ejecutar el script DDL: `@scripts/crear_tablas.sql` — crea las 6 tablas del esquema.
- Ejecutar el script de vistas: `@scripts/crear_vistas.sql` — crea las 5 VIEWS para reportes.
- Ejecutar datos de prueba: `@scripts/datos_iniciales.sql` — inserta categorías y usuario administrador.

Paso 2 — Configuración en NetBeans:

- Abrir NetBeans → File → Open Project → seleccionar carpeta del proyecto.
- Clic derecho sobre el proyecto → Properties → Libraries → Add JAR/Folder → seleccionar ojdbc8.jar.
- Verificar que ConexionDB.java tenga las credenciales correctas de la BD local.

Paso 3 — Compilación y Ejecución:

- Clic en el botón «Run Project» (F6) en NetBeans.
- La ventana de login debe aparecer. Ingresar con las credenciales del administrador creadas en el script SQL.

- Verificar en la consola de NetBeans que no hay errores de conexión JDBC.

8.3 Repositorio GitHub

<https://github.com/ahehao2-hub/QUIZ3CORTE.git>